Algorithms Summary

- **Common Problems & Best Algorithms**

1. Searching

- **Linear search (O(n))**
  - When: Data is **unsorted** and small.
  - Example: Searching for a name in an unsorted contact list.
- **Binary search (O(log n))**
  - When: Data is **sorted**.
  - Example: Finding a word in a dictionary.
- **Hashing (O(1))**
  - When: You need **fast lookups** and don't care about order.
  - Example: Checking if a user ID already exists.

---

2. Sorting

- **QuickSort (O(n log n) avg)**
  - When: General-purpose fast sort.
  - Example: Sorting an array of numbers.
- **MergeSort (O(n log n))**
  - When: You need **stable sorting** (order of equal elements preserved) or sorting linked lists.

- **HeapSort (O(n log n))**
  - When: You need guaranteed worst-case performance, or you're already using a heap.
- **Counting sort / Radix sort (O(n))**
  - When: Sorting integers in a **limited range**.
  - Example: Sorting exam scores from 0–100.

---

3. Finding Min/Max

- **Linear scan (O(n))**
  - When: One-time min/max needed.
- **Heap (O(1) for min/max, O(log n) for insert/delete)**
  - When: You need to keep track of min/max **as data changes**.
  - Example: Finding the current highest-priority task.

---

4. Graphs & Paths

- **Breadth-first search (BFS) (O(V+E))**
  - When: You need the shortest path in an **unweighted graph**.
  - Example: Finding the fewest moves in a chess puzzle.
- **Depth-first search (DFS) (O(V+E))**
  - When: You want to explore all paths or check connectivity.
  - Example: Checking if a path exists in a maze.
- **Dijkstra's algorithm (O(V log V + E))**
  - When: You need the shortest path in a **weighted graph** with non-negative weights.

    - Example: GPS navigation.
- *A search (like Dijkstra + heuristic)*
    - When: You need a **fast shortest path with direction**.
    - Example: Game AI pathfinding.

---

5. Dynamic Programming (DP)

- Used when problems overlap and can be broken into subproblems.
- Example problems:
    - Fibonacci numbers → avoid recalculating.
    - Knapsack problem → best combination of items.
    - Matrix pathfinding → min cost path.

---

6. String Problems

- **Naive substring search (O(nm))**
    - When: Data is small.
- **KMP / Rabin-Karp (O(n+m))**
    - When: Searching for substrings in large text.
- **Tries (prefix trees)**
    - When: You need autocomplete or fast prefix searches.

---

7. Other Useful Structures

- **Hash maps / Hash sets (O(1))** → Fast lookups, duplicates checking.
- **Balanced BST (AVL, Red-Black Tree, O(log n))** → Ordered data with fast search/insert/delete.
- **Priority queue / Heap (O(log n))** → Scheduling, always retrieving highest priority.
- **Union-Find (Disjoint Set, near O(1))** → Connected components, Kruskal's MST algorithm.

🔹 Rules of Thumb (When to Use What)

- **Need fast lookups?** → Hash Map / Hash Set.
- **Need ordered data?** → Balanced BST (like SortedSet).
- **Need shortest path?**
  - Unweighted → BFS.
  - Weighted → Dijkstra.
- **Need fast min/max tracking?** → Heap.
- **Working with sequences?**
  - Fixed random access → Array.
  - Lots of insertions/deletions in middle → Linked list.

🔹 Real-Life Examples

- Google Maps: **Dijkstra** for shortest path.
- Search engines: **Trie + Hashing + Inverted Index** for fast word search.
- Task schedulers: **Priority Queue (Heap)** for job ordering.

- Compression (ZIP, PNG): **Dynamic Programming + Greedy + Huffman coding**.

⚡ Summary

- Start simple → arrays, loops, hash maps.
- Use sorting + searching wisely.
- Graph problems → BFS/DFS, Dijkstra.
- DP for overlapping subproblems.
- Always ask: **Do I need order? Fast lookup? Min/max? Path?** → That tells you the right algorithm.